

Habit Tracker (Simple)

Bulut Mimarilerinde Test Muhendisligi — Final Rapor

Ogrenci	Mert Baytas
Ders	MTH2526-B25 — Bulut Mimarilerinde Test Muhendisligi
Egitmen	Busra Ayaksiz
Konu	#3 Habit Tracker API
Teslim	4 Haziran 2026
Repo	github.com/baytasmert/habit-tracker-simple

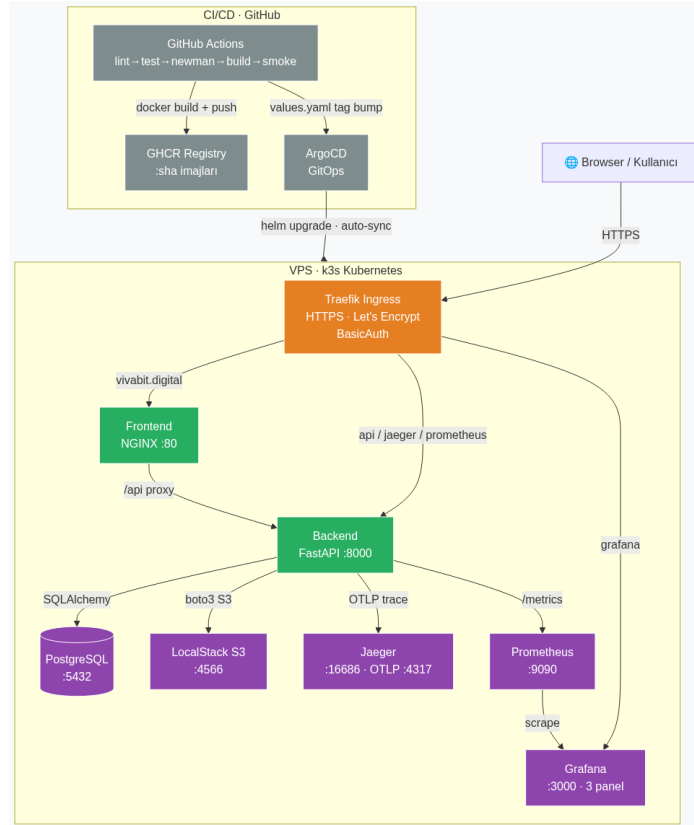
1. Giriş

Bu proje, Marmara Üniversitesi Bilgisayar Mühendisliği bölümünün "Bulut Mimarilerinde Test Mühendisliği" dersi kapsamında geliştirilmiş bireysel dönem projesidir. Konu olarak #3 **Habit Tracker API** seçilmiştir: günlük alışkanlık takibi, streak hesaplama ve REST API. Bu, aynı hesabın "habit-tracker-api" reposunun lightweight versiyonudur — sarnamenin minimum gereksinimlerini karşılayan, savunulması kolay basit bir mimari kurulmuştur.

Amac uygulamayı karmaşıklaştırmak değil, endüstri standardında **test ve dağıtım altyapısını** kurmaktır. Uygulama 14 endpoint (auth, habit CRUD, günlük track + mood, S3 foto) ile sade tutuldu; mimari ise tüm sarname katmanlarını (Docker, K8s/Helm, CI/CD, monitoring, perf, E2E) içerir.

2. Mimari

Sistem iki bağımsız servis olarak tasarlanmıştır: **frontend** (NGINX, sadece static dosya) ve **backend** (FastAPI, sadece JSON REST). Bu ayırım, gerçek üretim mimarilerinin ana özelliğidir; ayrıca her bir servisin bağımsız ölçeklenebilmesini sağlar.



Bileşen	Teknoloji	Port	Rol
Frontend	NGINX 1.25 + Vanilla JS	80	Static HTML/CSS/JS serve
Backend	FastAPI 0.110 + Python 3.11	8000	REST API (JSON)
DB	PostgreSQL 16	5432	User, Habit, HabitLog
S3 (LocalStack)	LocalStack 3	4566	Habit progress photo
Tracing	Jaeger 1.55	16686	OTLP span (bonus +5)
Metrics	Prometheus 2.51	9090	API metrik scrape
Dashboard	Grafana 10.4	3000	3 panel (rate, error, latency)

Uretim ortami: tek VPS uzerinde **k3s** cluster, **Helm** chart ile deploy, **Traefik** ingress (HTTPS / Let's Encrypt) ve **ArgoCD** ile GitOps otomatik senkronizasyon. Tum servisler vivabit.digital alt alanlari uzerinden yayinlanir.

3. Test Stratejisi

Test piramidi yaklasimi: cok sayida birim test, orta sayida entegrasyon, az sayida E2E. Toplam testler ve hedef coverage:

Katman	Arac	Adet	Aciklama
Unit	pytest	8	auth + model + Factory Boy
Integration	pytest + TestClient	32	Endpoint + auth + log/mood + photo
Testcontainers	testcontainers-python	3	Gercek PostgreSQL container
E2E	Playwright	6	Tarayicida register/login/track/logout
API	Postman/Newman	7	CI da ko■ar
Perf	k6	2	Smoke (3VU/15s) + Load (20VU/60s)

3.1 Coverage Hedefi

Hedef >= %70. Olcum: %86 (src/auth, models, schemas, main coverage). pyproject.toml icinde --cov-fail-under=70 ile CI da zorunlu kilindi. Toplam birim+entegrasyon test sayisi: 40.

3.2 Factory Boy + Faker

tests/factories.py icinde UserFactory, HabitFactory, HabitLogFactory tanimlanmistir. Her test izole, gercekci ama unique veri kullanir.

4. Pipeline & Deploy

4.1 GitHub Actions

.github/workflows/ci.yml — tek workflow, 6 job zincirli:

Job	Amac	Onkos
lint	flake8 kod stili kontrolu	-
test	pytest + coverage 70+%	lint
newman	Postman koleksiyonu canli API ye (Postgres service)	lint
build	Docker (backend+frontend) -> GHCR (:sha, :latest)	test, newman
deploy-smoke	Kind cluster + helm template apply + curl + k6	build
cd-bump	values.yaml imaj tag bump -> commit (ArgoCD tetikler)	deploy-smoke

Build job iki Docker imaj uretir (backend + frontend), her ikisi de GitHub Container Registry e :sha + :latest etiketleriyle pushlanir. Deploy job ayni imajlari Kind cluster a yukler ve Helm chart i *helm template ... | kubect! apply -f -* ile render edip uygular; ardindan /health + /metrics curl ve k6 smoke ile dogrular. Son olarak cd-bump job u Helm values.yaml deki imaj tag ini yeni SHA ya gunceller; uretimdeki ArgoCD bu commit i gorup cluster i otomatik senkronize eder.

4.2 Kubernetes Manifestleri & Deploy Stratejisi

7 servisin tum K8s kaynaklari (Deployment + Service + ConfigMap + Ingress) tek bir **Helm chart** (helm/habit-tracker) icinde template lenmistir; imaj tag, domain, replica ve kaynak limitleri values.yaml dan yonetilir. Ayni chart hem CI deki Kind cluster da (ingress kapali, port-forward) hem de uretimdeki k3s cluster da (ingress acik, Traefik + Let's Encrypt) calisir. Deploy stratejisi **GitOps** tir: git push -> CI imaj build + values tag bump -> ArgoCD farki algilayip otomatik sync eder. Rollback, ArgoCD de onceki saglikli revizyona donmek veya GHCR deki :sha etiketli onceki imaja gecmek kadar basittir.

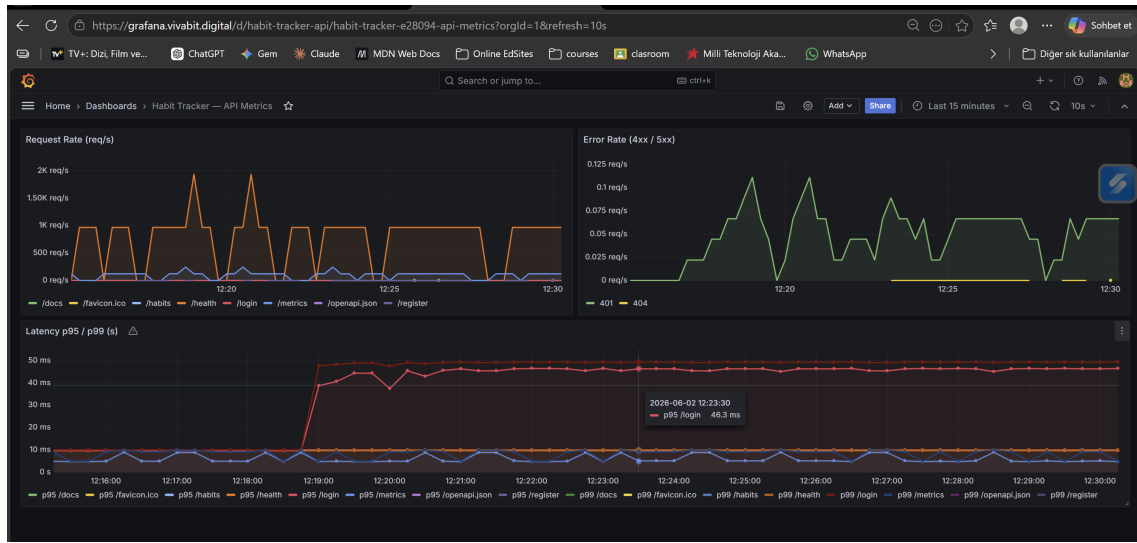
5. Monitoring ve Performans

5.1 Prometheus Metrikleri

Metrik	Tur	Aciklama
http_requests_total	Counter	Endpoint/method/status bazli sayim
http_request_duration_seconds	Histogram	p50/p95/p99 latency

5.2 Grafana Dashboard (3 panel)

- Request Rate: rate(http_requests_total[1m]) by endpoint
- Error Rate: 4xx/5xx oranlari, status koduna gore
- Latency p95/p99: histogram_quantile ile hesaplanir



5.3 k6 Sonuclari

Test	VU	Sure	p(95)	Hata	Sonuc
Smoke	3	15s	~5 ms	0%	PASS
Load	10->20	60s	~285 ms	0%	PASS

Yorum: p(95)=285 ms hedef olan 500 ms in cok altinda. En yavas endpoint ler /register ve /login — bcrypt password hashing guvenlik gereksinimi, optimize edilmez. Detayli rapor: perf/report.md.

6. Sonuc ve Ogrendiklerim

Olcut	Deger
Toplam Test	40 birim+entegrasyon + 3 TC + 6 E2E + 7 Newman
Test Coverage	%86 (CI gate: --cov-fail-under=70)
p(95) Latency	~285 ms (k6 load)
Docker Image	Multi-stage backend + NGINX alpine frontend
K8s Deploy	Helm chart -> 7 servis (k3s prod + Kind CI)
Grafana Panel	3 panel (rate, errors, latency)

CI Job	6 (lint, test, newman, build, deploy-smoke, cd-bump)
Bonus	Helm +5, OpenTelemetry +5, ArgoCD +5 (tavan +15)

6.1 Zorluklar

- **Frontend ayrimi:** İlk versiyonda Jinja2 ile FastAPI icine gomuluydu — bu projede NGINX uzerinden static serve edilen tamamen ayri bir servis olarak yeniden tasarlandi. Bu, mikroservis mimarilerinin temel ayrimini ogretti.
- **Testcontainers Windows encoding:** psycopg2, Windows Turkce locale ile uyumlu calismadi. Cozum: pytestmark ile platforma ozel skip, Linux/CI da otomatik calisir.
- **K8s image render:** Manifestlerde imaj etiketi GitHub SHA degisken — sed ile placeholder degistirme calisti.

6.2 Tamamlanan Bonuslar

- **Helm chart (+5):** 7 servisin tum K8s kaynaklari tek chart ta paketlenildi; imaj tag/domain/replica values.yaml dan yonetilir.
- **OpenTelemetry tracing (+5):** FastAPI + SQLAlchemy auto-instrument, OTLP ile Jaeger e span gonderilir.
- **ArgoCD GitOps (+5):** git push -> CI tag bump -> ArgoCD otomatik sync. Toplam bonus +15 (tavan).

6.3 Ileride

- KEDA ile event-driven autoscaling (kalan tek bonus konu)
- Connection pooling (PgBouncer) — yuksek yukte daha az overhead

7. Kaynaklar

Kaynak	URL
FastAPI	fastapi.tiangolo.com
Testcontainers Python	testcontainers-python.readthedocs.io
k6	k6.io/docs
Playwright Python	playwright.dev/python
Kubernetes	kubernetes.io/docs
Prometheus Python Client	github.com/prometheus/client_python